

Data Movement Pipeline using Azure Data Factory and Azure DevOps

Table of Contents

1. Project Statement
2. Project Overview & Objectives
3. Prerequisites
4. Azure Resources Used
5. Implementation Steps
6. Verification & Output
7. CI/CD Pipeline Setup
8. Conclusion & Learnings
9. Architecture Diagram & Workflow Explanation
10. Challenges & Recommendations

1. Project Statement

The aim of this project is to design and implement a **data movement pipeline** using **Azure Data Factory (ADF)** that can reliably transfer data from a simulated on-premises **Azure SQL Database** to **Azure Synapse Analytics**, a cloud-based data warehouse solution. This pipeline not only automates the ETL process but also integrates with **Azure DevOps** to enable version control and CI/CD deployment. The solution demonstrates real-world cloud data engineering practices, including pipeline orchestration, secure data transfer, monitoring, and collaborative development workflows.

2. Project Overview & Objectives

2.1 Overview

Modern data engineering requires seamless integration between on-premises sources and cloud analytics platforms. In this project, structured data stored in an Azure SQL Database simulates an on-premises environment. **Azure Data Factory** orchestrates the movement of this data into a cloud-based warehouse (**Synapse Analytics**), ensuring that the data is transformed and stored efficiently. **Azure DevOps** complements this workflow by managing version control, enabling collaboration among team members, and supporting CI/CD deployment practices for automated updates.

2.2 Objectives

The primary objectives of this project are to:

- Provision and configure the necessary **Azure resources** to support a scalable and secure data pipeline.
- Build an **ADF pipeline** capable of extracting, transforming, and loading data from the SQL Database to ADLS Gen2 and Synapse Analytics.
- Validate the accuracy and completeness of data transfers through testing and query verification.
- Integrate **ADF with Azure DevOps** for source control, collaborative development, and versioned deployment of pipeline artifacts.
- Initiate a **CI/CD pipeline** to facilitate automated deployment, with partial implementation due to current limitations in DevOps organization access.

Additionally, the project emphasizes **best practices**, such as structured data storage, modular pipeline design, error handling, and logging for maintainability and traceability.

3. Prerequisites

Prior to implementation, the following requirements must be fulfilled:

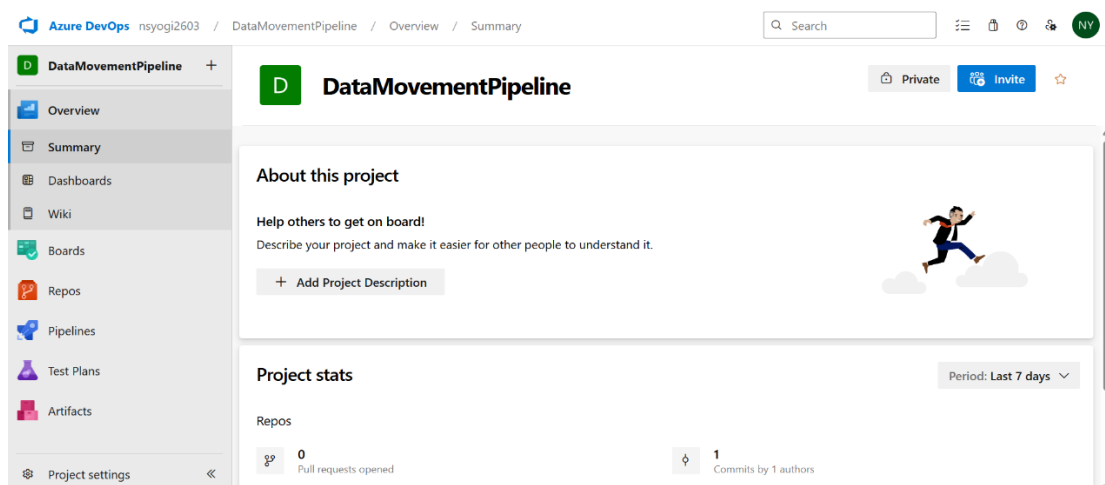
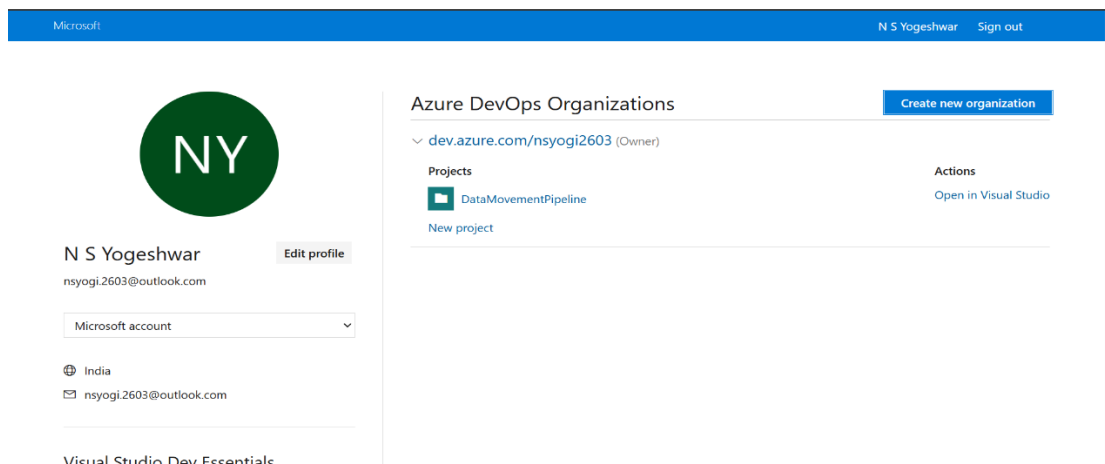
An **Azure subscription** with permissions to create and manage **ADF, Synapse, SQL Database, and Storage accounts** is essential. Similarly, an **Azure DevOps account** with a project and repository setup is required for version control and CI/CD integration. Users must also have **basic knowledge of ETL concepts, data pipelines, Git version control, and Azure Portal navigation** to implement and troubleshoot the workflow effectively. Access to tools such as **Azure Synapse Studio** and **SQL Server Management Studio (SSMS)** is recommended for data validation and querying.

4. Azure Resources Used

The solution utilizes a combination of Azure services to implement a reliable data movement pipeline. These resources include a **resource group** (rg-hexaver-adf-synapse-dev) to logically organize all assets, **Azure Data Lake Storage Gen2** for structured data storage with raw and curated containers, and **Azure Synapse Analytics** for cloud-based data warehousing. The **Azure SQL Database** simulates an on-premises data source, while **Azure Data Factory** orchestrates the data flow. Finally, **Azure DevOps** provides version control, collaborative development, and the foundation for a CI/CD deployment pipeline.

Note on Subscription Usage

"Since my personal Azure subscription had already expired, I carried out this task using my college's Azure account. This allowed me to successfully complete the setup and proceed with the deployment without interruptions."



5. Implementation Steps

5.1 Creating the Resource Group

The first step in the implementation involved creating a **resource group**. This serves as a logical container for all Azure resources related to the project, ensuring centralized management and consistent access control. By grouping resources together, it is easier to manage permissions, monitor costs, and maintain an organized environment that can scale as the project grows.

The screenshot shows the 'Create a resource group' page in the Azure portal. The breadcrumb navigation at the top reads 'Home > Resource groups >'. The page title is 'Create a resource group' with a close button (X) on the right. Below the title are three tabs: 'Basics' (selected), 'Tags', and 'Review + create'. A descriptive text block explains that a resource group is a container for related resources. Below this, there are three input fields: 'Subscription' (a dropdown menu showing 'Azure for Students'), 'Resource group name' (a text box containing 'rg-hexaware-synapse-dev'), and 'Region' (a dropdown menu showing '(Asia Pacific) Central India'). At the bottom of the page, there are three buttons: 'Previous' (disabled), 'Next' (disabled), and 'Review + create' (active).

5.2 Setting up Azure Data Lake Storage Gen2

A **Data Lake Storage Gen2** account was provisioned to store the pipeline data. Hierarchical namespace was enabled to support folder-level operations, improving performance and manageability. Two primary containers were created: raw for the ingestion of unprocessed data and curated for storing transformed and validated datasets. This separation ensures a clean ETL process, making it easier to track data lineage and perform incremental updates.

The screenshot shows the 'Create a storage account' page in the Azure portal. The breadcrumb navigation at the top reads 'Home > Storage accounts >'. The page title is 'Create a storage account' with a close button (X) on the right. Below the title are several tabs: 'Basics', 'Advanced', 'Networking', 'Data protection', 'Encryption', 'Tags', and 'Review + create' (selected). A link for 'View automation template' is present. The 'Basics' section contains a table of configuration options: Subscription (Azure for Students), Resource group (rg-hexaware-adf-synapse-dev), Location (Central India), Storage account name (sthexawareyn), Primary service (Standard), Performance (Standard), and Replication (Locally-redundant storage (LRS)). The 'Advanced' section contains a table of settings: Enable hierarchical namespace (Enabled), Enable SFTP (Disabled), Enable network file system v3 (Disabled), Allow cross-tenant replication (Disabled), Access tier (Hot), and Enable large file shares (Enabled). The 'Security' section contains a table of settings: Secure transfer (Enabled), Blob anonymous access (Disabled), and Allow storage account key access (Enabled). At the bottom, there are three buttons: 'Previous' (disabled), 'Next' (disabled), and 'Create' (active). A footer note reads 'Default Az. Microsoft Entra authentication is Disabled.' and a 'Give feedback' link is on the right.

5.3 Creating Azure Synapse Analytics Workspace

An **Azure Synapse Analytics workspace** was deployed to act as the destination for structured, transformed data. Integration with ADLS Gen2 was verified in **Synapse Studio** to ensure that the pipeline could access the storage account securely. This setup allows querying and analysis of the data once it has been processed, supporting analytics and reporting workflows.

Home > Azure Synapse Analytics >

Create Synapse workspace

Validation succeeded

BasicsSecurityNetworkingTagsReview + create

Product Details

Azure Synapse Analytics workspace by Microsoft
[Terms of use](#) | [Privacy policy](#)

Serverless SQL est. cost/TB
\$0.00 USD

Terms

By clicking Create, I (a) agree to the legal terms and privacy statement(s) associated with the Marketplace offering(s) listed above; (b) authorize Microsoft to bill my current payment method for the fees associated with the offering(s), with the same billing frequency as my Azure subscription; and (c) agree that Microsoft may share my contact, usage and transactional information with the provider(s) of the offering(s) for support, billing and other transactional activities. Microsoft does not provide rights for third-party offerings. For additional details see [Azure Marketplace Terms](#).

Basics

Subscription

Resource group

Region

Workspace name

Data Lake Storage Gen2 account

Data Lake Storage Gen2 file system

Managed resource group

Role assignments

Azure for Students

rg-hexaware-adt-synapse-dev

Central India

(new) syn-hex-devops

[https://sthexwrmv.dfs.core.windows.net](#)

curated

None

The Storage Blob Data Contributor role will be assigned on the specified Data Lake Storage Gen2 account to the workspace managed identity.

Create

< Previous

Next >

Download a template for automation

5.4 Provisioning Azure SQL Database

To simulate an on-premises environment, an **Azure SQL Database** was provisioned as the data source. Sample tables with realistic data were created, complete with primary keys and constraints to maintain data integrity. This database provided a reliable foundation for pipeline testing, enabling validation of ETL processes and troubleshooting of schema and type mismatches during the development phase.

Home > SQL databases >

Create SQL Database

Microsoft

SQL Database Hyperscale: Low price, high scalability, and best feature set. [Learn more](#)

Project details

Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription *

Resource group *

Azure for Students

rg-hexaware-adt-synapse-dev

Create new

Database details

Enter required settings for this database, including picking a logical server and configuring the compute and storage resources

Database name *

Server *

sqldbonprem

onpremsvr (Central India)

Create new

Want to use SQL elastic pool?

Workload environment

No

Development

Production

Default settings provided for Development workloads. Configurations can be modified as needed.

Review + create

Next: Networking >

Home > SQL databases >

Configure

Feedback

Compute tier

Provisioned - Compute resources are pre-allocated. Billed per hour based on vCores configured.

Serverless - Compute resources are auto-scaled. Billed per second based on vCores used.

Compute Hardware

Select the hardware configuration based on your workload requirements. Availability of compute optimized, memory optimized, and confidential computing hardware depends on the region, service tier, and compute tier.

Hardware Configuration

Standard series (Gen5)

up to 80 vCores, up to 240 GB memory

[Change configuration](#)

Max vCores

Min vCores

2.02 GB MIN MEMORY

3 GB MAX MEMORY

Auto-pause delay

The database automatically pauses if it is inactive for the time period specified here, and automatically resumes when database activity recurs. Alternatively, auto-pausing can be disabled.

Enable auto-pause

Days

Hours

Minutes

0

1

0

Data max size (GB)

Apply

Microsoft Azure

Search resources, services, and docs (Ctrl+K)

Copilot

nsyogi.z603@outlook.c...
SUBMIT FEEDBACK

Home > sqldb_onprem (onpremservers/sqldb_onprem) > onpremsvr

onpremsvr | Networking

Search

Feedback

Overview

Activity log

Access control (IAM)

Tags

Quick start

Diagnose and solve problems

Resource visualizer

Settings

Data management

Security

Networking

Microsoft Defender for Cloud

Transparent data encryption

Identity

Auditing

Intelligent performance

Monitoring

Automation

Help

Public access

Private access

Connectivity

Public network access

Public endpoints allow access to this resource through the internet using a public IP address. An application or resource that is granted access with the following network rules still requires proper authorization to access this resource. [Learn more](#)

Public network access

Disable

Selected networks

Connections from the IP addresses configured in the Firewall rules section below will have access to this database. By default, no public IP addresses are allowed. [Learn more](#)

Virtual networks

Allow virtual networks to connect to your resource using service endpoints. [Learn more](#)

+ Add a virtual network rule

Rule

Virtual network

Subnet

Address range

Endpoint status

Resource group

Subscription

State

Firewall rules

Allow certain public internet IP addresses to access your resource. [Learn more](#)

+ Add your client IP4 address

+ Add a firewall rule

Rule name

Start IP4 address

End IP4 address

ClientAddress_2025-0-25_11-42-19

152.57.63.223

152.57.63.223

Save

Discard

Add or remove descriptions by pressing Ctrl+K (Ctrl+K)

5.5 Setting up Azure Data Factory

Azure Data Factory (ADF) was provisioned to orchestrate the data movement. The pipeline was designed to follow **modular principles**, enabling reuse of activities, datasets, and linked services. Error handling and logging mechanisms were added to monitor activity runs and capture potential failures. The design also incorporated parameterization to allow flexibility for different environments and datasets.

The screenshot shows the 'Create Data Factory' wizard in the Azure portal, specifically the 'Review + create' step. The breadcrumb navigation at the top indicates the path: Home > Data factories > Create Data Factory. The wizard has tabs for Basics, Git configuration, Networking, Advanced, Tags, and Review + create. A link to 'View automation template' is present. The 'Basics' section displays the following details: Subscription (Azure for Students), Resource group (rg-hexaware-adf-synapse-dev), Name (hex-df-decops), Region (Central India), and Version (V2). The 'Networking' section shows 'Connect via' set to 'Public endpoint'. At the bottom, there are 'Previous', 'Next', and 'Create' buttons, along with a 'Give feedback' link.

5.6 Configuring Azure DevOps for Version Control

To maintain version control and enable collaborative development, an **Azure DevOps organization, project, and repository** were created. ADF was linked to this repository, allowing all pipeline artifacts to be tracked using **Git**. This setup ensures that changes are versioned, supports rollback if needed, and facilitates collaboration among multiple developers working on the same pipeline.

The screenshot displays the 'Configure a repository' dialog box within the Azure Data Factory interface. The left sidebar shows the navigation menu with 'Git configuration' selected under 'Source control'. The main area of the dialog is titled 'Configure a repository' and includes a 'Configure' button. The right pane contains the configuration settings: 'Default Directory' is set to 'Default Directory (8e9ef295-66c3-4d69-ac46-3ce30848a1bf)'; 'Select repository' is chosen over 'Use repository link'; 'Azure DevOps organization name' is 'yogeshwar-devops'; 'Project name' is 'DataMovementPipeline'; 'Repository name' is 'DataMovementPipeline'; 'Collaboration branch' is 'main'; 'Publish branch' is 'adf_publish'; 'Root folder' is '/'; 'Custom comment' is checked; 'Import existing resources' is checked; 'Import existing resources to repository' is checked; and 'Import resource into this branch' is 'main'. At the bottom, there are 'Apply', 'Back', and 'Cancel' buttons.

5.7 Creating Linked Services

Linked services were defined in ADF to establish secure connections between the source and sink. The source linked service connects to the Azure SQL Database using managed identities or stored credentials, while the sink linked service connects to ADLS Gen2. These connections ensure data can flow securely and efficiently between the source and target systems.

New linked service
 Azure SQL Database [Learn more](#)

Encrypt 
Mandatory 

Trust server certificate  ☐

Host name in certificate

Additional connection properties

[+ New](#)

Annotations

[+ New](#)

> Parameters

[+ Annotations](#)

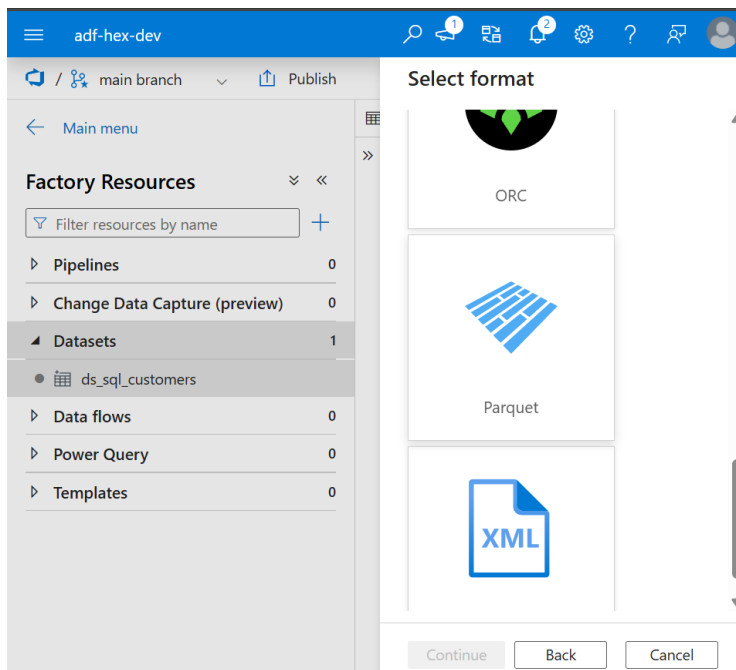
[Create](#) [Back](#)

 Connection successful

 Test connection [Cancel](#)

5.8 Creating Datasets

The source and sink datasets were created to define the data structures for pipeline operations. The **source dataset** points to the SQL tables, while the **sink dataset** writes data into **Parquet format** in ADLS Gen2. Parquet was chosen for its columnar storage capabilities, which improve query performance and reduce storage costs. Schema mapping was defined to ensure compatibility between source and target fields.



5.9 Building and Debugging the Pipeline

A **Copy Data activity** pipeline was constructed to perform the ETL operation. The pipeline included source-to-sink mapping, incremental load configuration, and error handling. A **debug run** was executed to validate the pipeline, confirming successful data transfer, correct mapping of fields, and adherence to the desired folder structure in ADLS.

main branch Publish Preview experience Off

Main menu

Factory Resources

Filter resources by name

- Pipelines 1
 - pipeline1
- Change Data Capture (preview) 0
- Datasets 2
 - ds_adls_gen2
 - ds_sql_customers
- Data flows 0
- Power Query 0
- Templates 0

pipeline1

Add trigger

Properties

General Related

Name *

pipeline1

Description

Annotations

+ New

main branch Validate all Preview experience Off

ds_sql_customers ds_adls_g

Successfully saved

Successfully saved pl_copy_sql_to_adls (Pipeline).

Saved Validate copy runtime

General Source Sink Mapping Settings User properties

ng Clear Reset Delete

Destination	Type
→ CustomerID	123 INT_32
→ FirstName	abc UTF8
→ LastName	abc UTF8
→ Email	abc UTF8
→ City	abc UTF8

adf-hex-dev

main branch Pipeline validation output

ds_sql_customers

Save as template

Copy data

Parameters Variable

Pipeline run ID 6a58

Pipeline status ✓ S

All status

Showing 1 - 1 of 1 iter

Activity name ↑↓

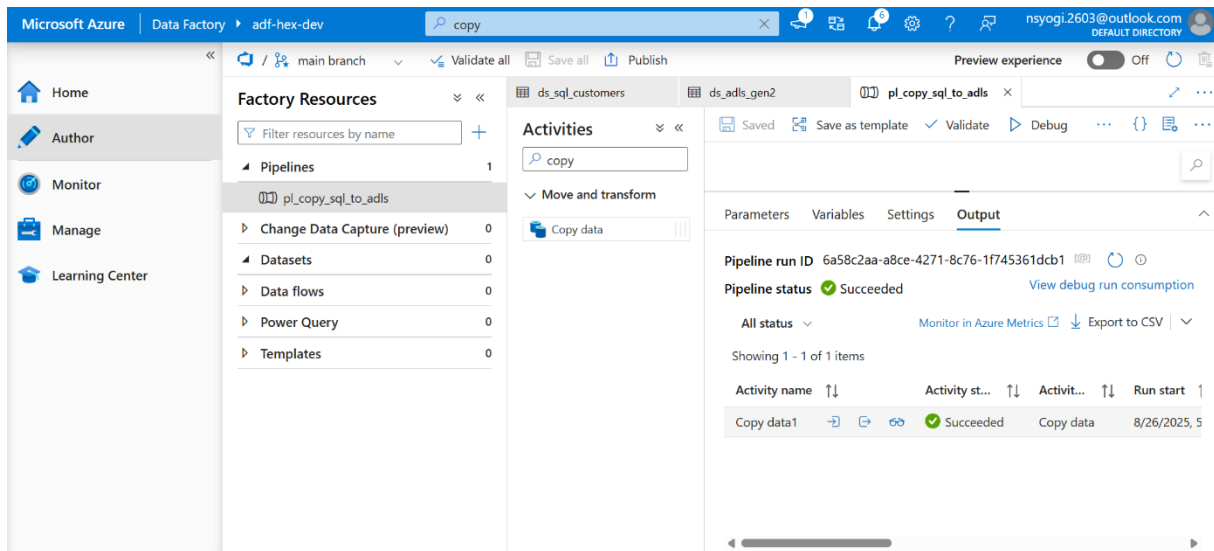
Copy data1

Close



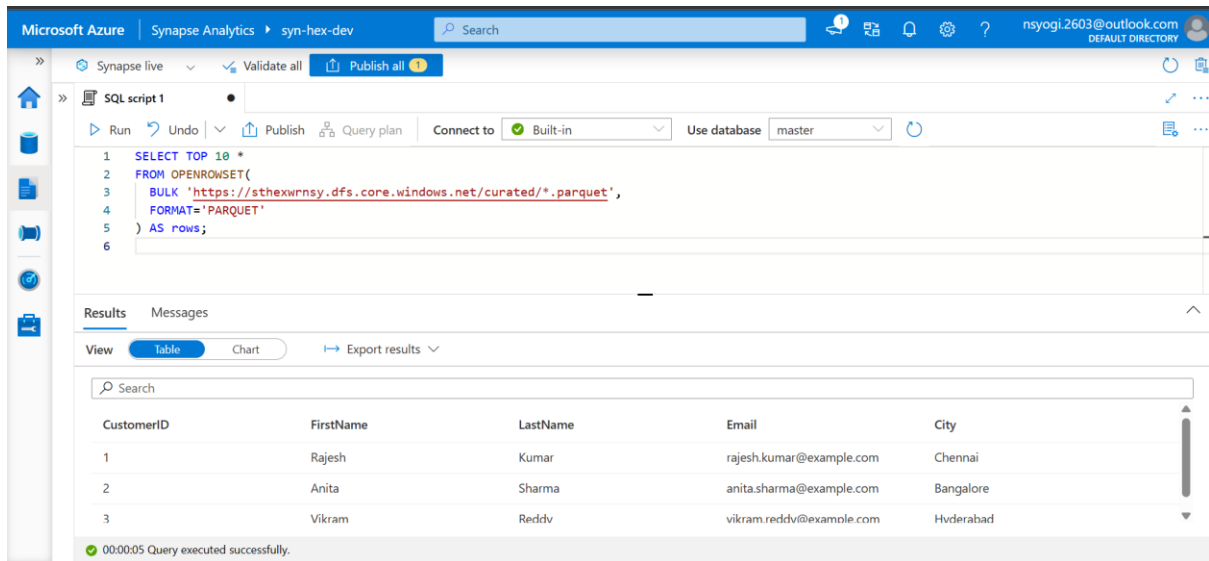
Your pipeline has been validated.

No errors were found.



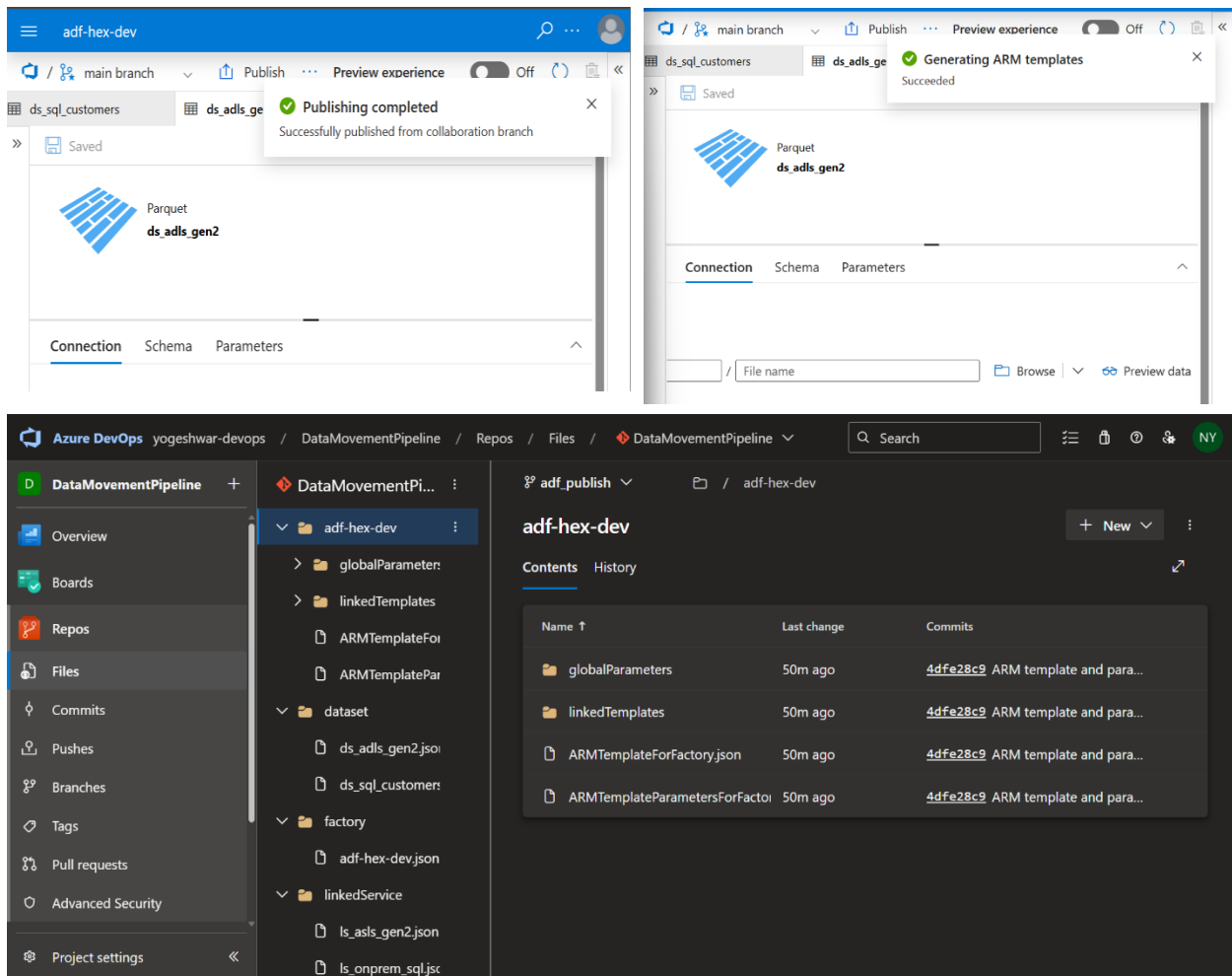
5.10 Verification via Synapse Studio

Once data was loaded into Synapse Analytics, it was verified using **OPENROWSET queries**. This step ensured that the data in the warehouse matched the source, both in terms of record count and field values. Validation confirmed the correctness of the pipeline and the readiness of the dataset for downstream analytics.



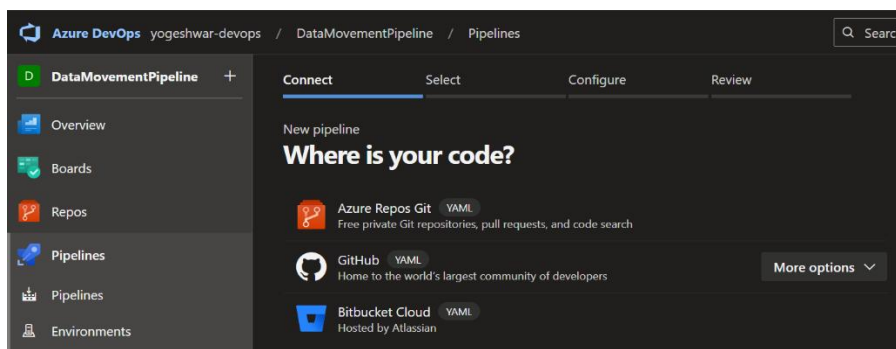
6. Verification & Output

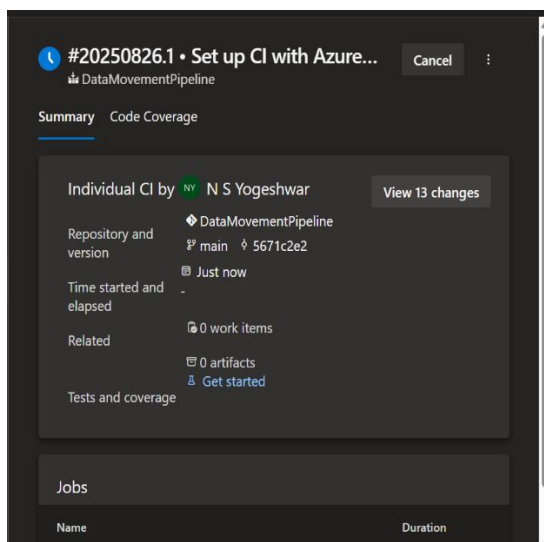
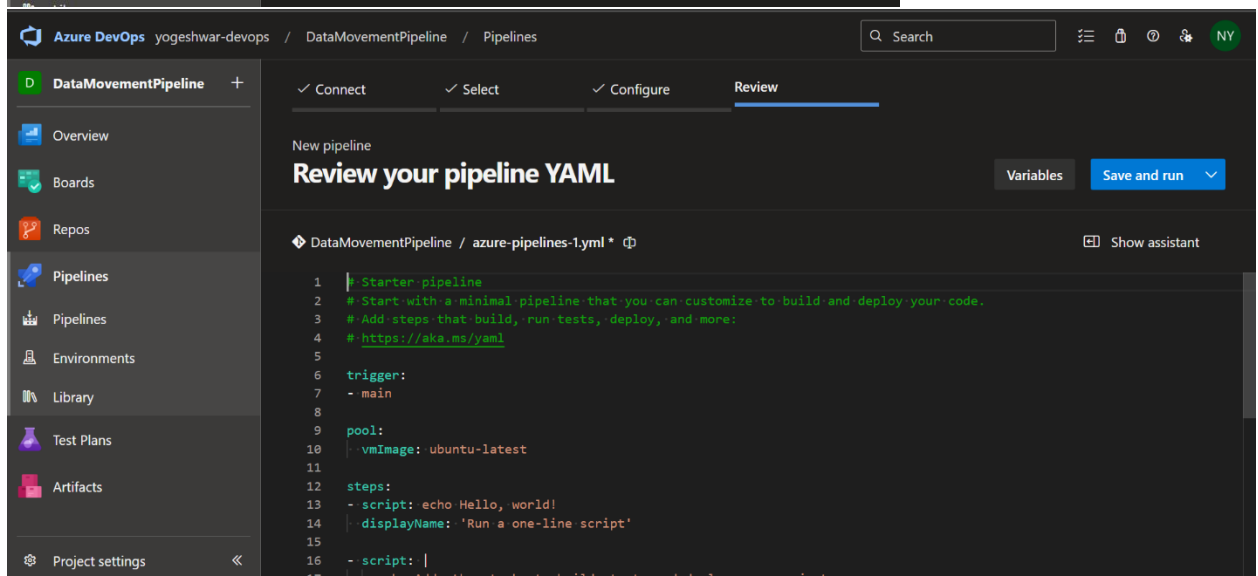
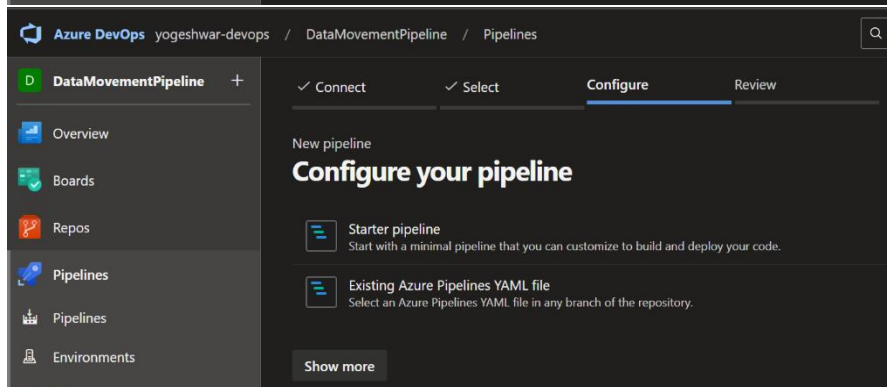
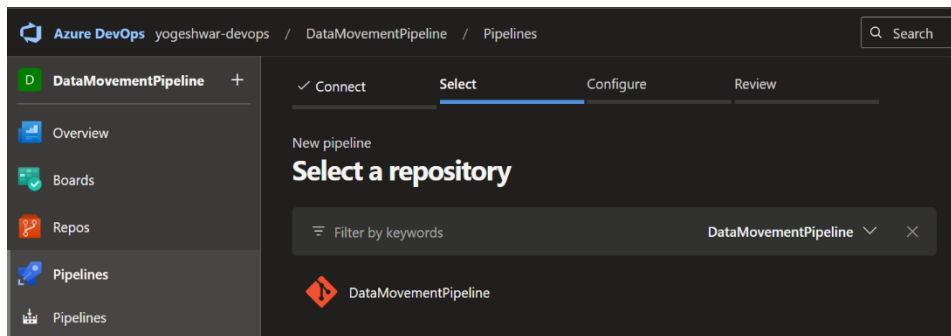
The pipeline executed successfully, transferring data from the SQL Database to ADLS and Synapse Analytics. Output data was validated for accuracy, completeness, and formatting. All pipeline artifacts were published and stored in the **adf_publish** folder in Azure DevOps, ensuring proper version control and deployment readiness. The process demonstrated robust ETL orchestration, proper error handling, and efficient data transformation.



7. CI/CD Pipeline Setup

A **service connection** was configured in Azure DevOps to enable automated deployment. The **release pipeline approach** was initiated to support continuous integration and deployment of pipeline artifacts. While deployment is currently pending due to parallelism restrictions in the private DevOps organization, the setup lays the foundation for fully automated CI/CD workflows. Future steps include integrating approval gates, monitoring, and rollback mechanisms for production readiness.





8. Conclusion & Learnings

Through this project, a fully functional **data movement pipeline** was built using Azure Data Factory and integrated with Azure DevOps for CI/CD readiness. The solution demonstrates end-to-end ETL processes, secure cloud integration, and modern DevOps practices. Key learnings include the importance of modular pipeline design, proper logging and monitoring, data validation techniques, and effective use of cloud storage and warehouse solutions. This project also highlighted real-world challenges, such as connection management, data type compatibility, and deployment limitations in private DevOps organizations, providing valuable experience for enterprise-level cloud projects.